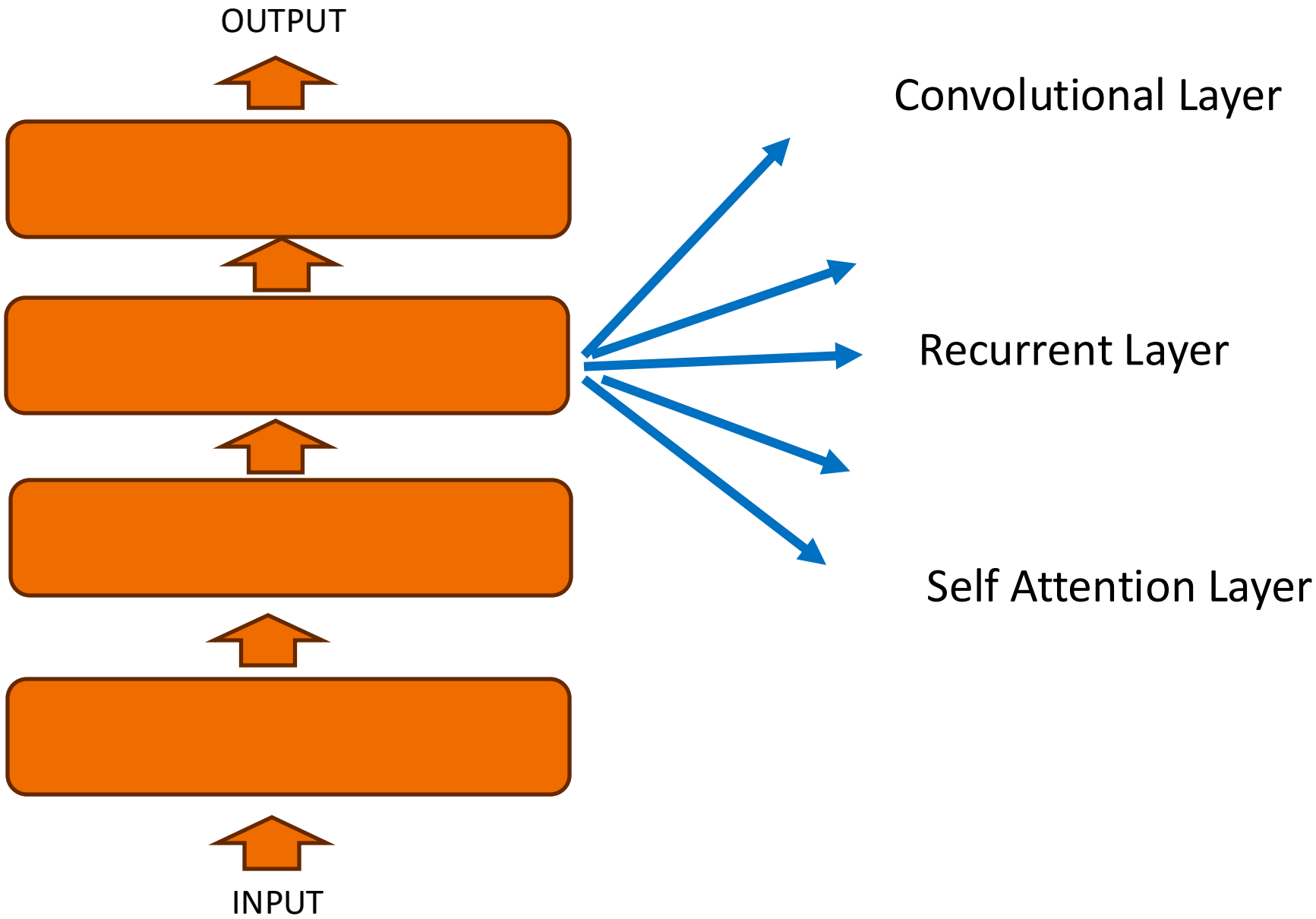
Self-Attention

DL and Different Types of Layers

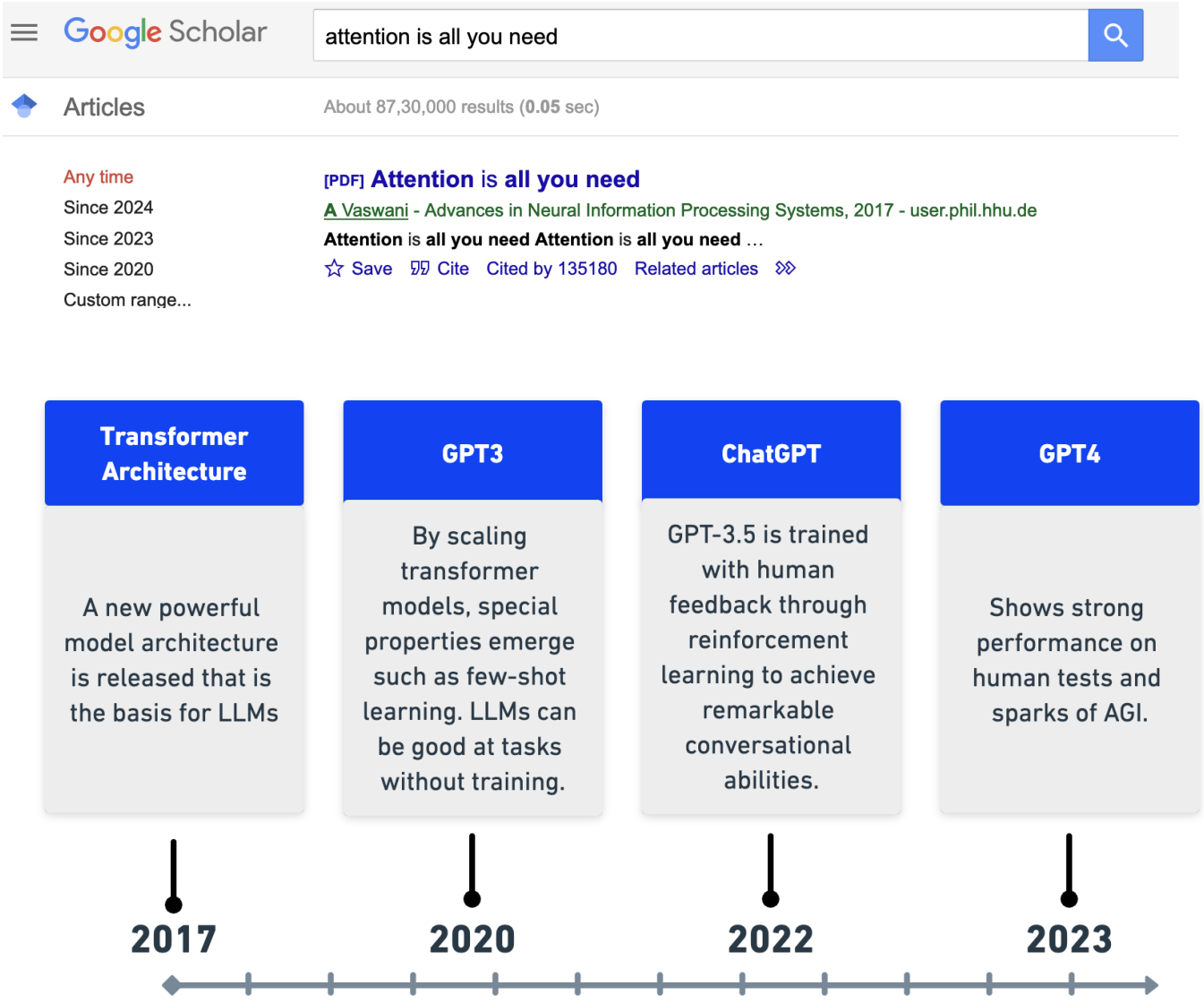


Convolutional Layer

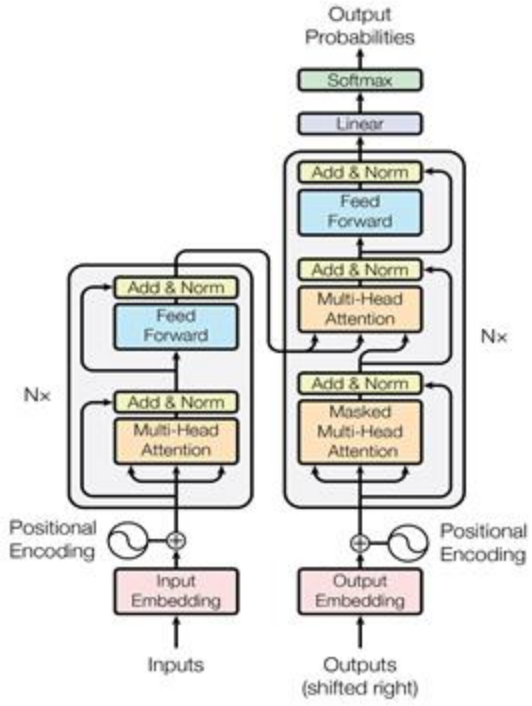
Recurrent Layer

Self Attention Layer

Transformers



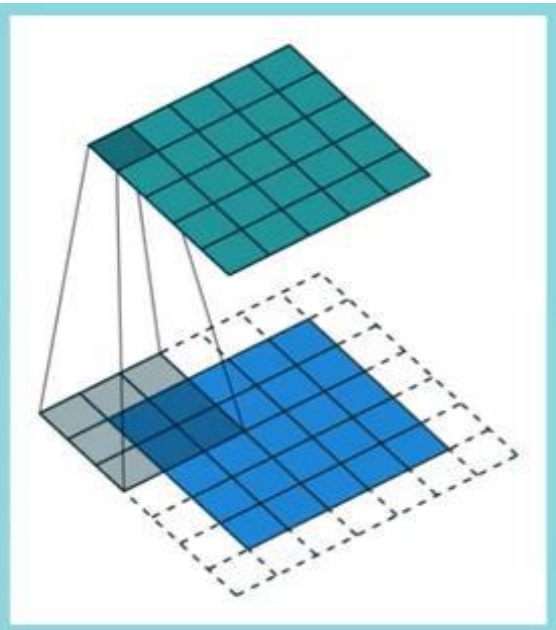
Task: Translate from one language to another

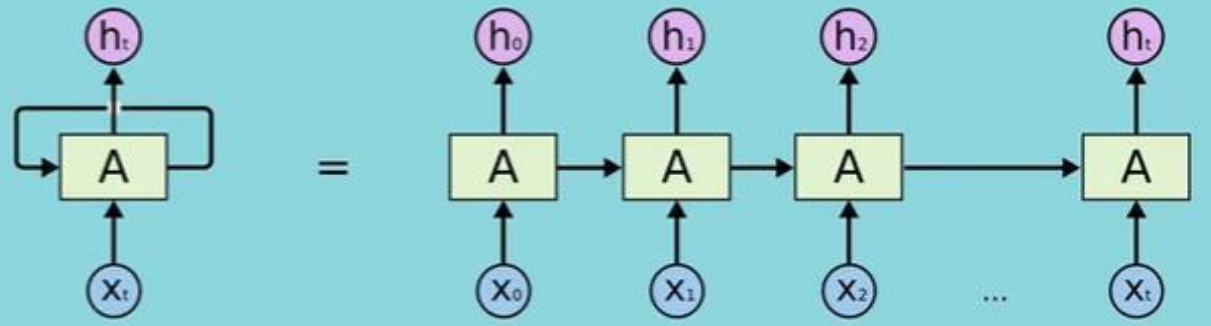


Transformer Encoder
Transformer Decoder

Attention is all you need (2017)

CNN, RNN and Transformers





Pro: Compute in Parallel
Con: Finite Horizon

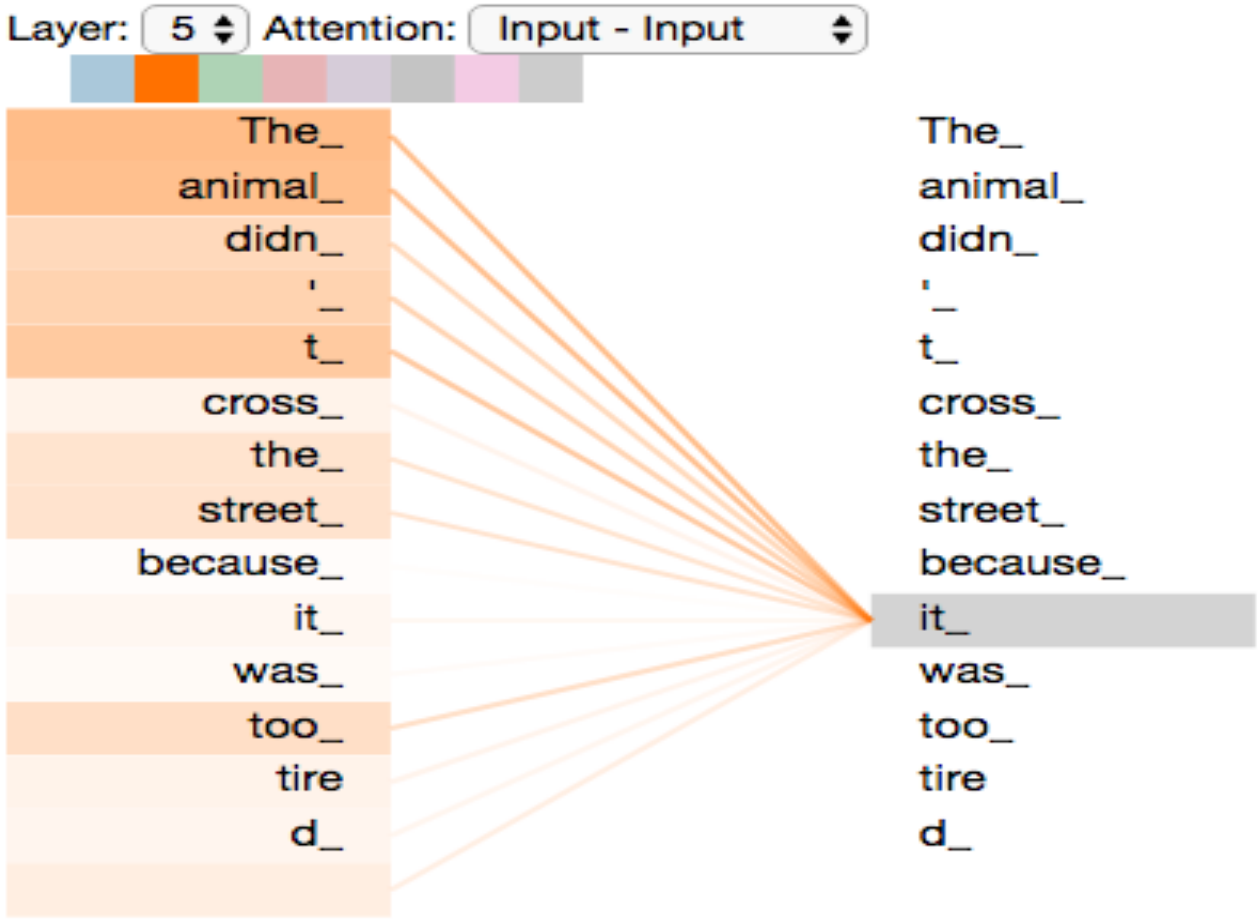
Pro: Infinite Horizon
Con: Compute Serially only

Attention

Blank Slide

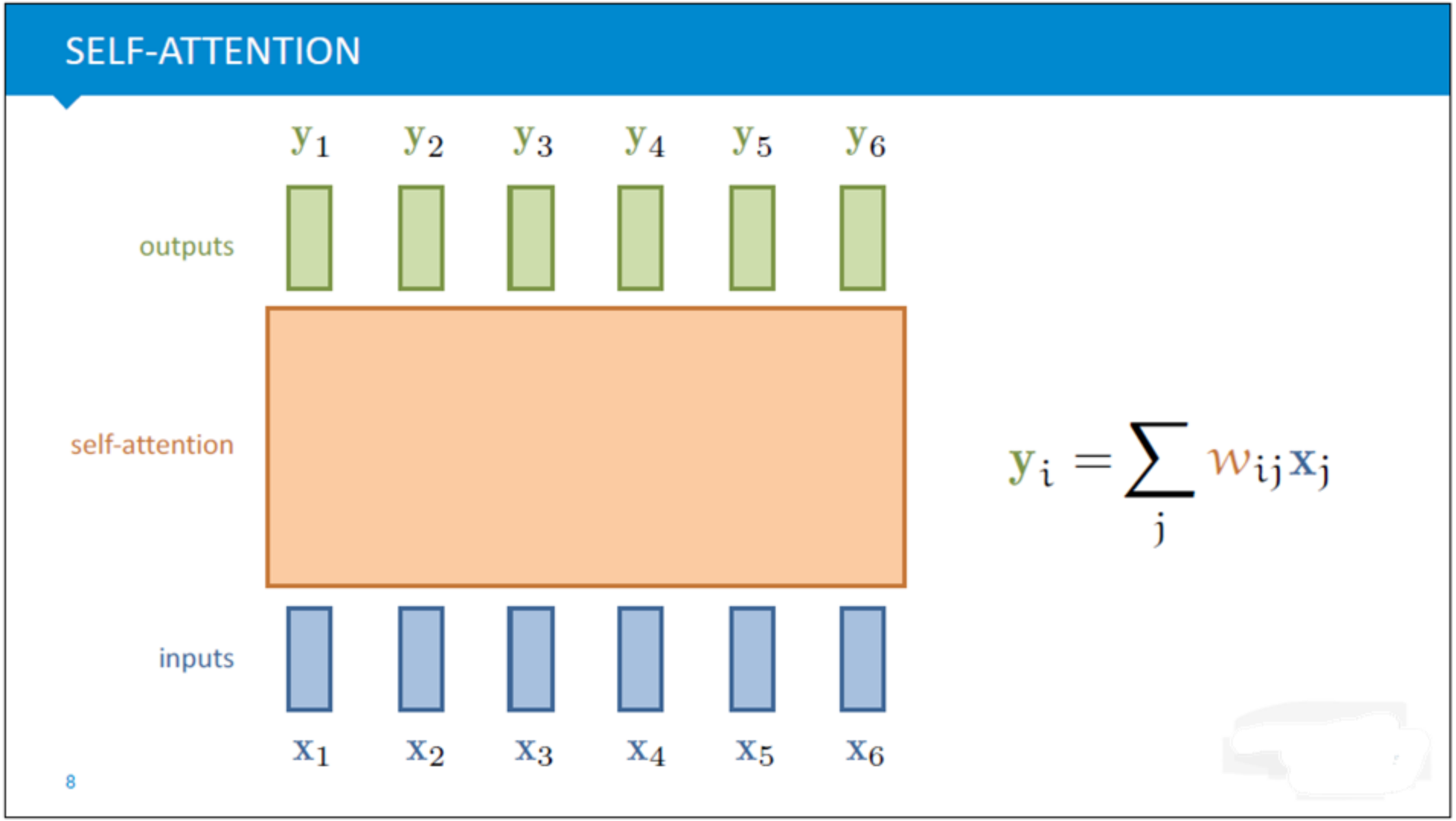
I. Simple Self Attention

Self Attention



1. **The animal** *did not cross the road because* **it** *was too tired.*
2. *The animal did not cross* **the road** *because* **it** *was too wide.*

SA: Computation



$$y_i = \sum_j w_{ij} x_j$$

$$w'_{ij} = x_i^T x_j$$

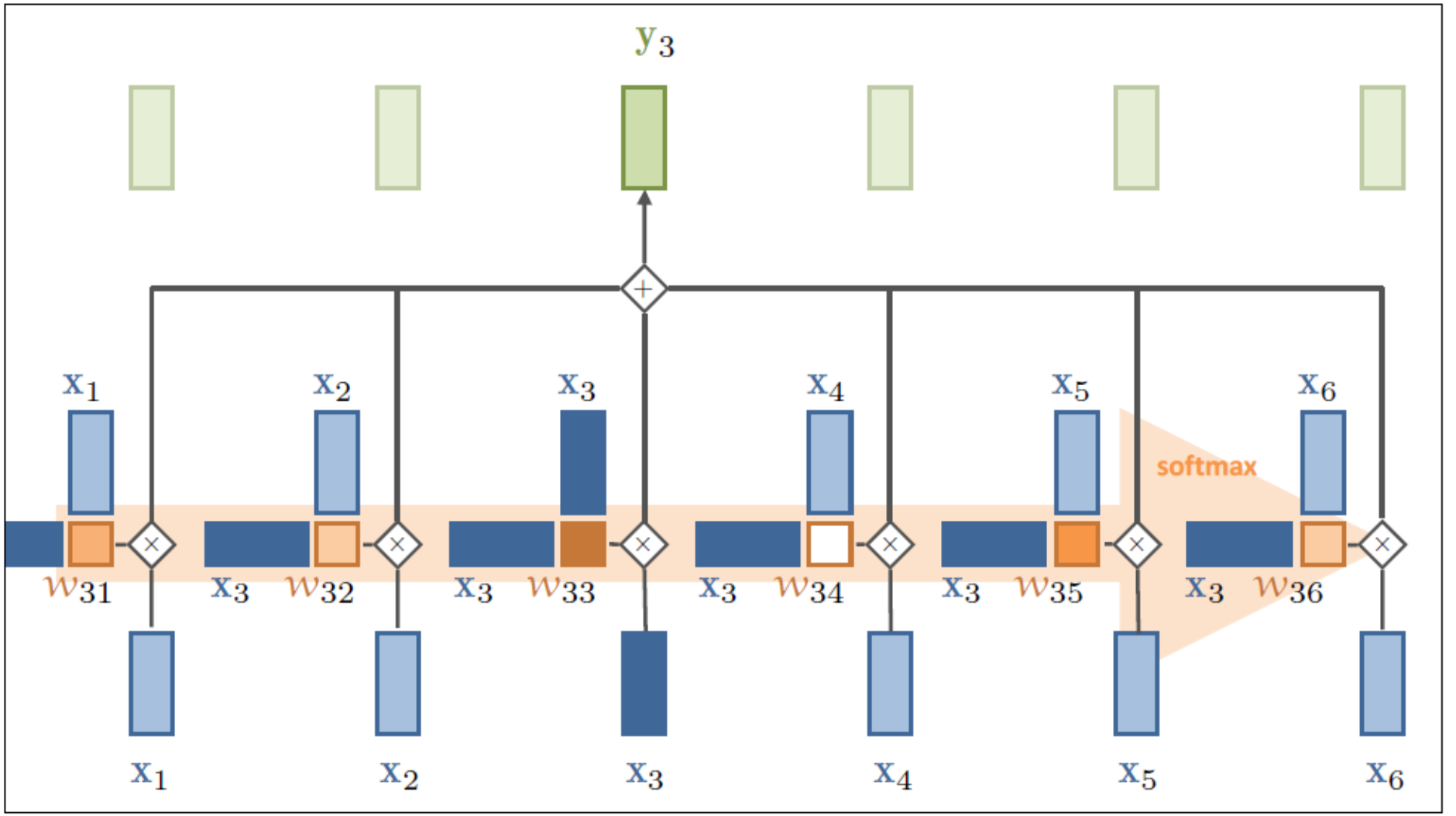
$$w_{ij} = \frac{\exp w'_{ij}}{\sum_i \exp w'_{ij}}$$

Blank Slide

SA: Observations

- Input and Output:
  - Vectors are of same in number and dimension
  - Set of vectors (in simple form). Add position, if required.
- Every output vector is influenced by “all” input vectors.
- “W” is not learned here. It is computed from the input/data.

SA: Computation



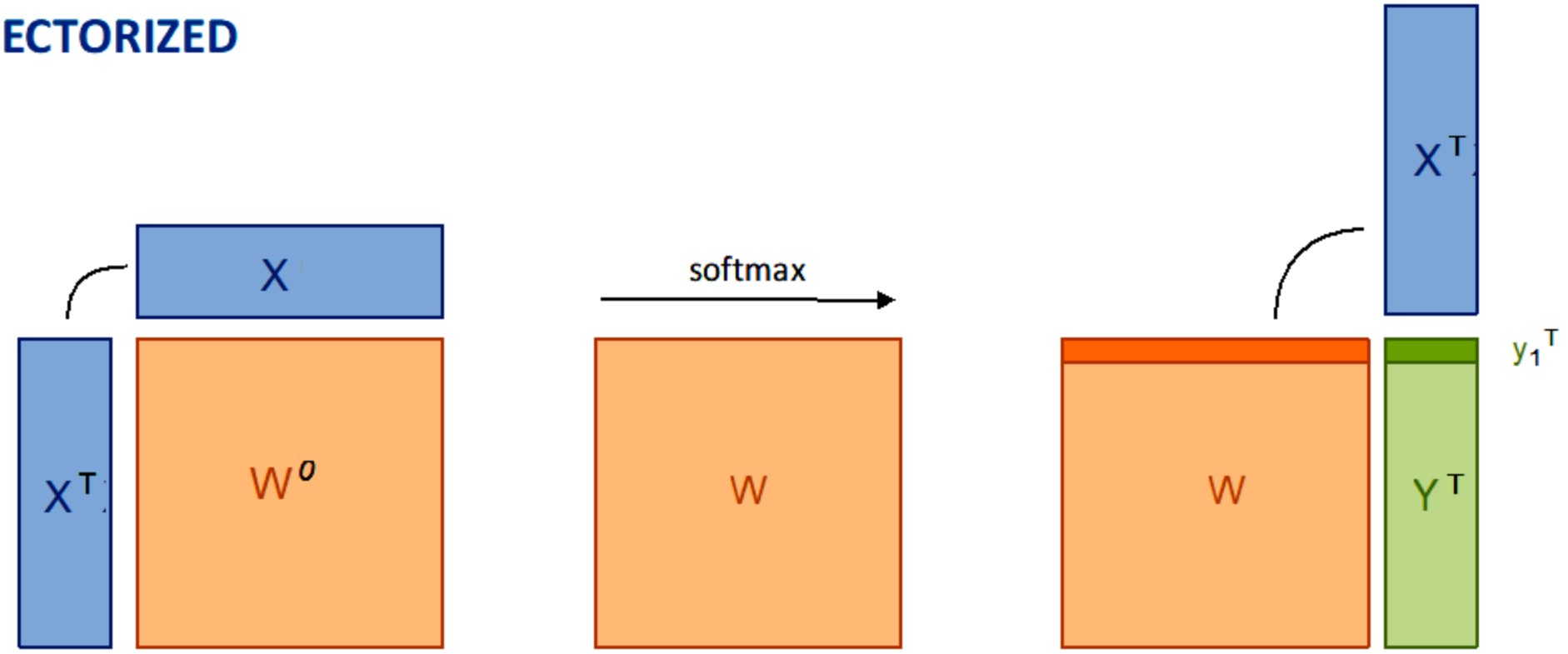
$$y_i = \sum_j w_{ij} x_j$$

$$w'_{ij} = x_i^T x_j$$

$$w_{ij} = \frac{\exp w'_{ij}}{\sum_i \exp w'_{ij}}$$

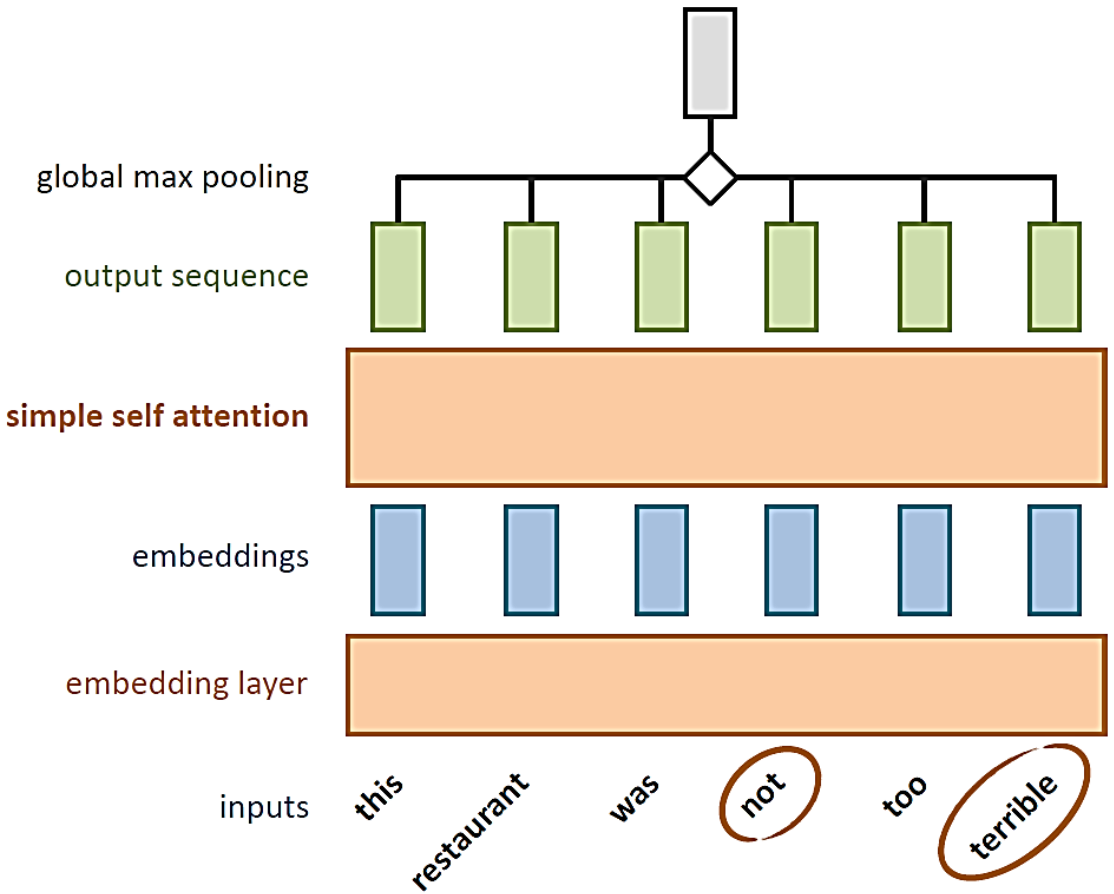
As matrix operations

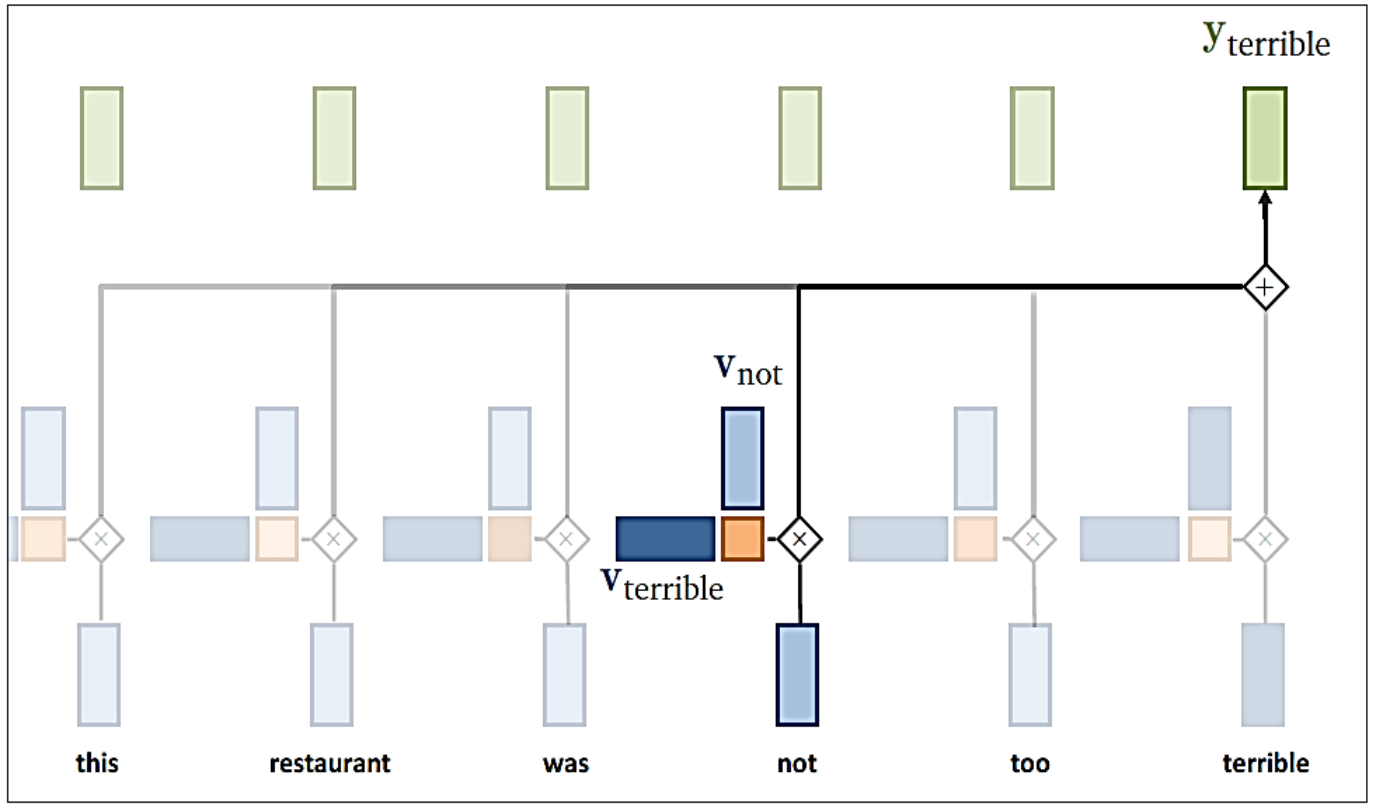
VECTORIZED



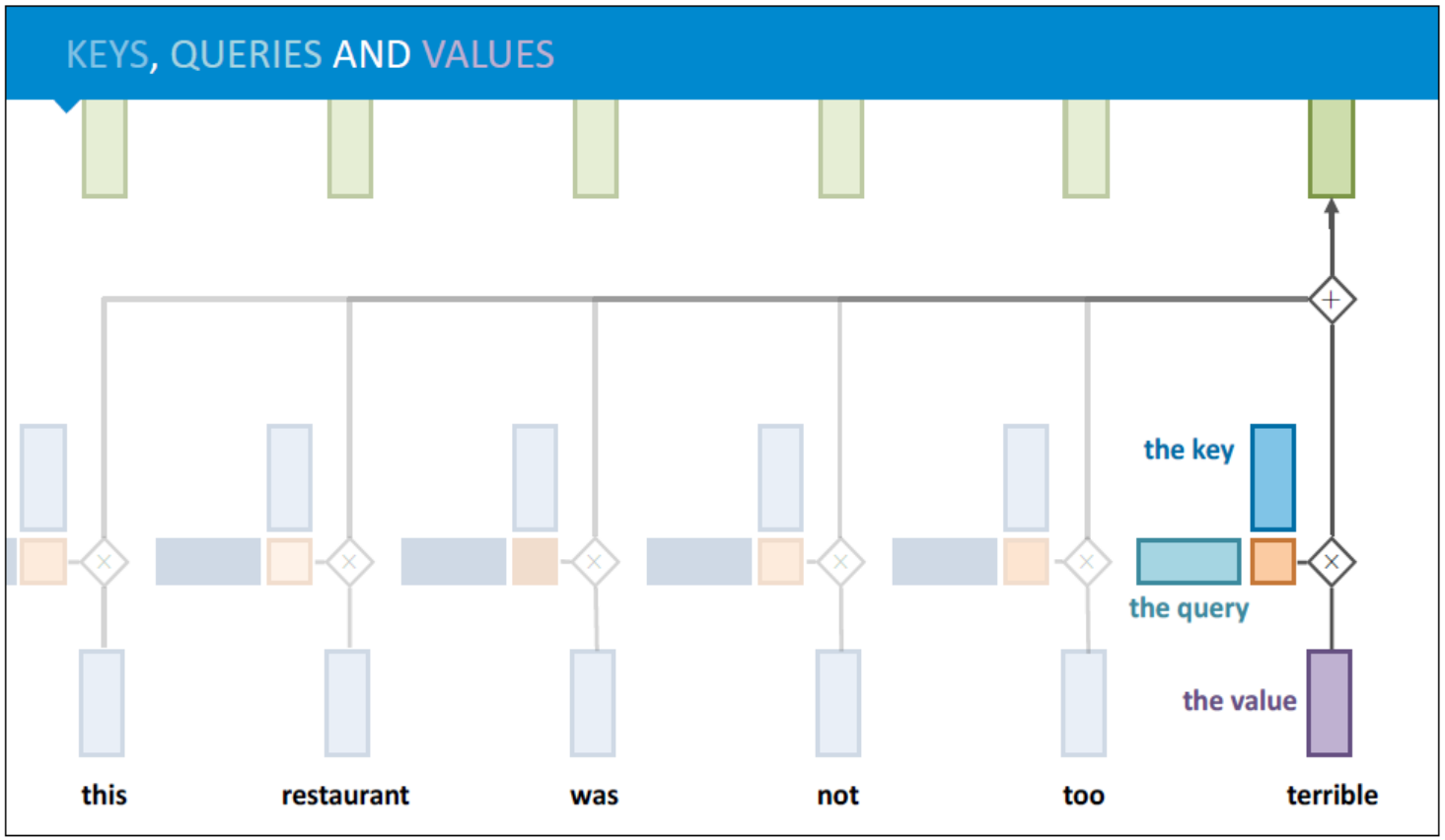
Blank Slide

Example


global max pooling
output sequence
simple self attention
embeddings
embedding layer
inputs
this
restaurant
was
not
too
terrible


$y_{terrible}$
v_{not}
$v_{terrible}$
this
restaurant
was
not
too
terrible

Query, Key and Value



In each self attention computation, every input vector occurs in three distinct roles:

- **the value**: the vector that is used in the weighted sum that ultimately provides the output
- **the query**: the input vector that corresponds to the current output, matched against every other input vector.
- **the key**: the input vector that the query is matched against to determine the weight.

Blank Slide

Example

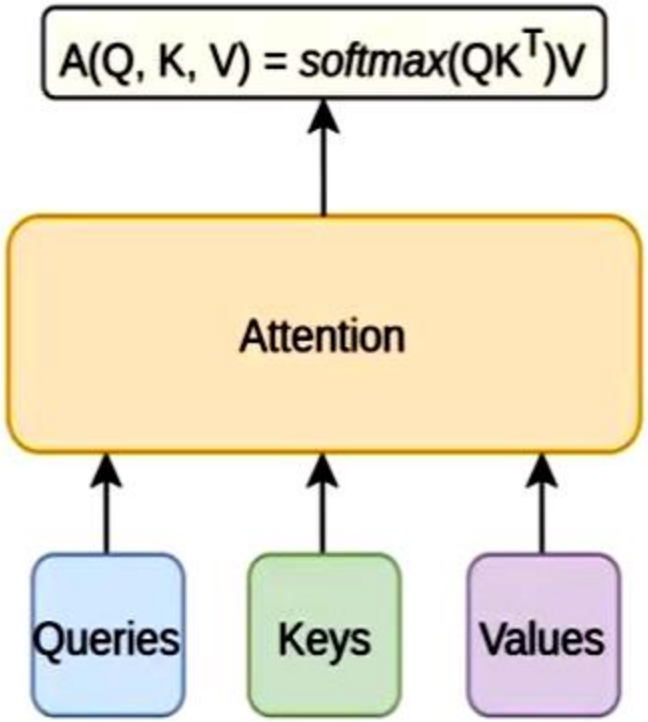
We ask: what is the value of $\begin{bmatrix}2\\0\\3\end{bmatrix}$? $\quad y = \sum_{j=1}^{N} (k_j^T q) \cdot v_j$

keys	values	weight
$\begin{bmatrix}0.5\\0.5\\0\end{bmatrix}$	1	$k_1^T \cdot q = [0.5, 0.5, 0] \cdot \begin{bmatrix}2\\0\\3\end{bmatrix} = 0.5 * 2 + 0.5 * 0 + 0 * 3 = 1$
$\begin{bmatrix}0.5\\-0.5\\0\end{bmatrix}$	3	$k_2^T \cdot q = [0.5, -0.5, 0] \cdot \begin{bmatrix}2\\0\\3\end{bmatrix} = 0.5 * 2 + (-0.5) * 0 + 0 * 3 = 1$
$\begin{bmatrix}0\\0\\1\end{bmatrix}$	5	$k_3^T \cdot q = [0, 0, 1] \begin{bmatrix}2\\0\\3\end{bmatrix} = 0 * 2 + 0 * 0 + 1 * 3 = 3$

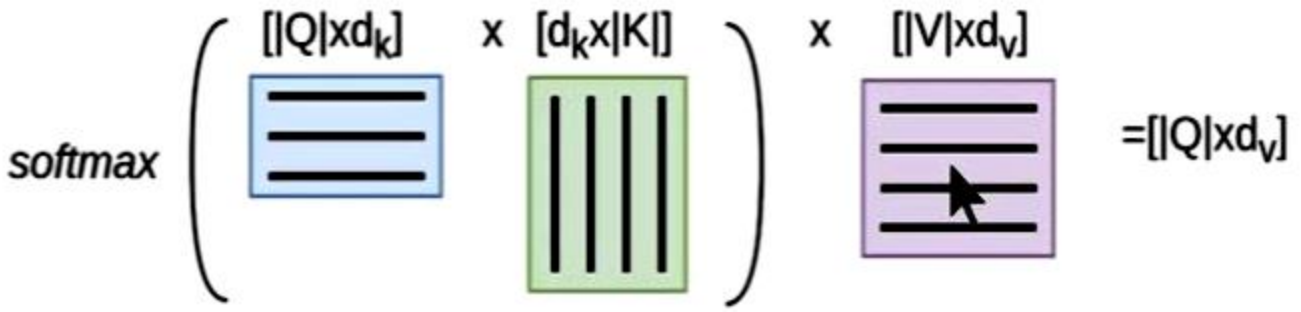
$= 1 * 1 + 1 * 3 + 3 * 5 = 19$

Blank Slide

What is Attention?



$$y_i = \sum_{j=1}^{N} \left(k_j^T q_i\right) \cdot v_j$$



Blank Slide

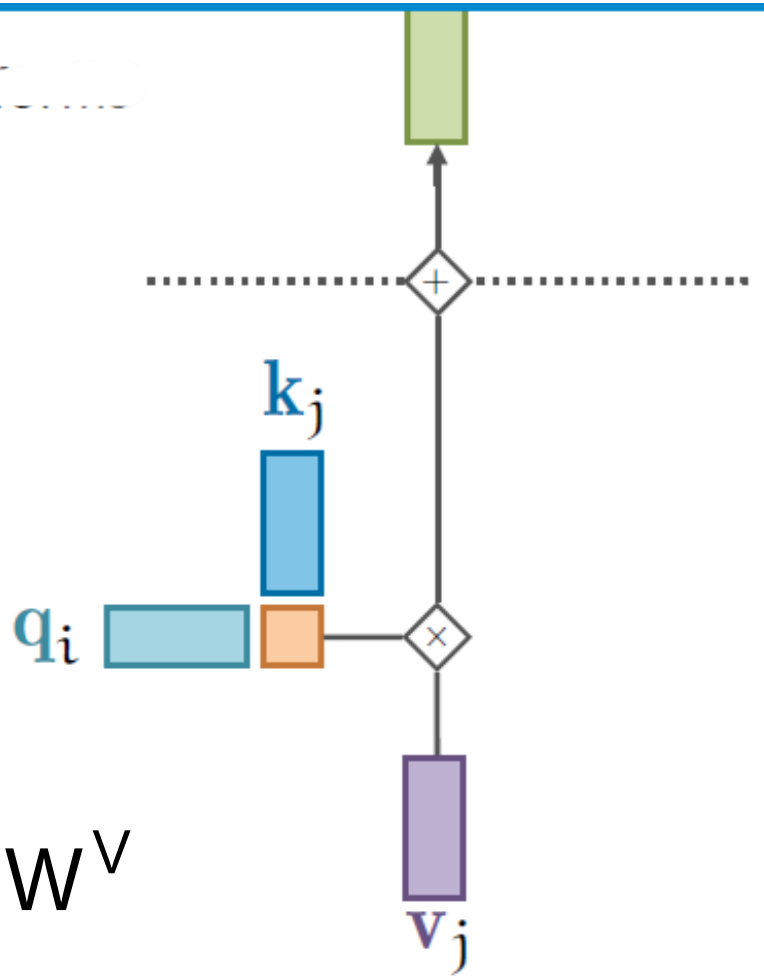
II. (Weighted) Self Attention

Learnable “Query”, “Key” and “Value”

$$\mathbf{k}_i = \mathbf{K}\mathbf{x}_i + \mathbf{b}_k$$

$$\mathbf{q}_i = \mathbf{Q}\mathbf{x}_i + \mathbf{b}_q$$

$$\mathbf{v}_i = \mathbf{V}\mathbf{x}_i + \mathbf{b}_v$$



$q_i = x_i W^Q; k_i = x_i W^K; v_i = x_i W^V$

Blank Slide

Self Attention

One query per input vector

Inputs:

Input vectors: X (Shape: $N_X \times D_Q$)

Key matrix: $W_K (Shape: D_X \times D_Q)$

Value matrix: $W_V (Shape: D_X \times D_V)$

Query matrix: W_Q (Shape: $D_Q \times D_Q$)

Computation:

Query Vectors $Q = XW_!$

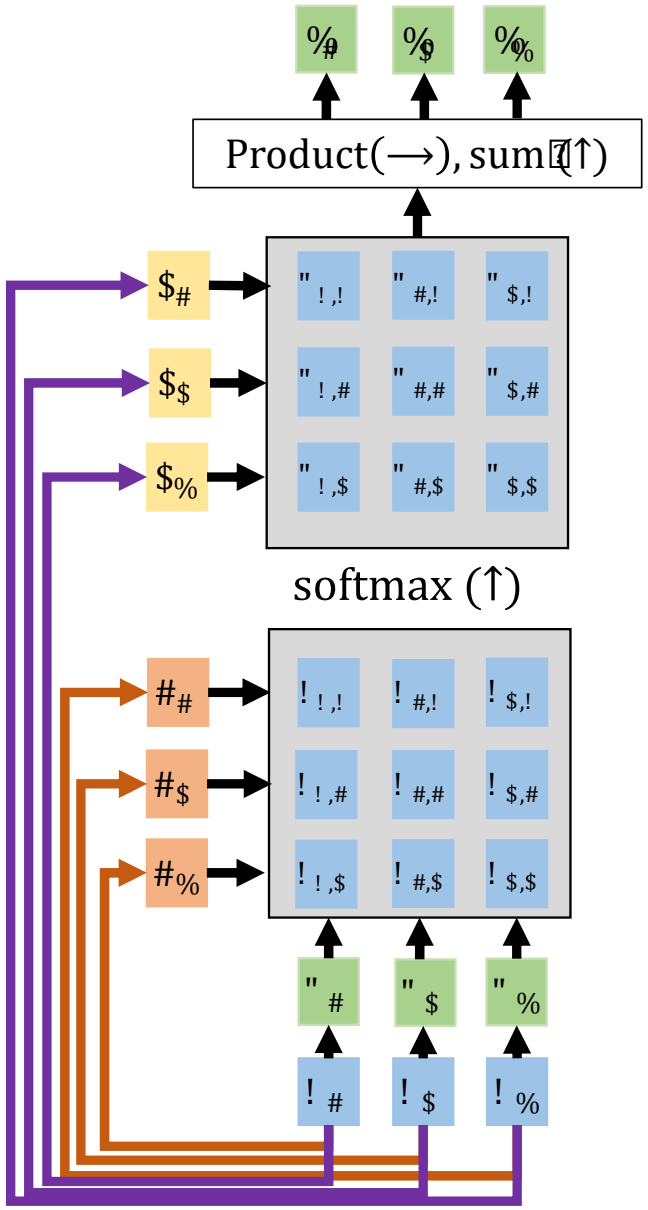
Key vectors: $K = XW_) \ (Shape: N_" \times D_!)$

Value Vectors: $V = XW_* \ (Shape: N_" \times D_*)$

Similarities: $E = \frac{! \ " \ !}{\sqrt{\$ \ "}} \ (Shape: N_" \times N_") E_{\%\&} = (Q_{\%} \cdot K_')/\sqrt{D_(}$

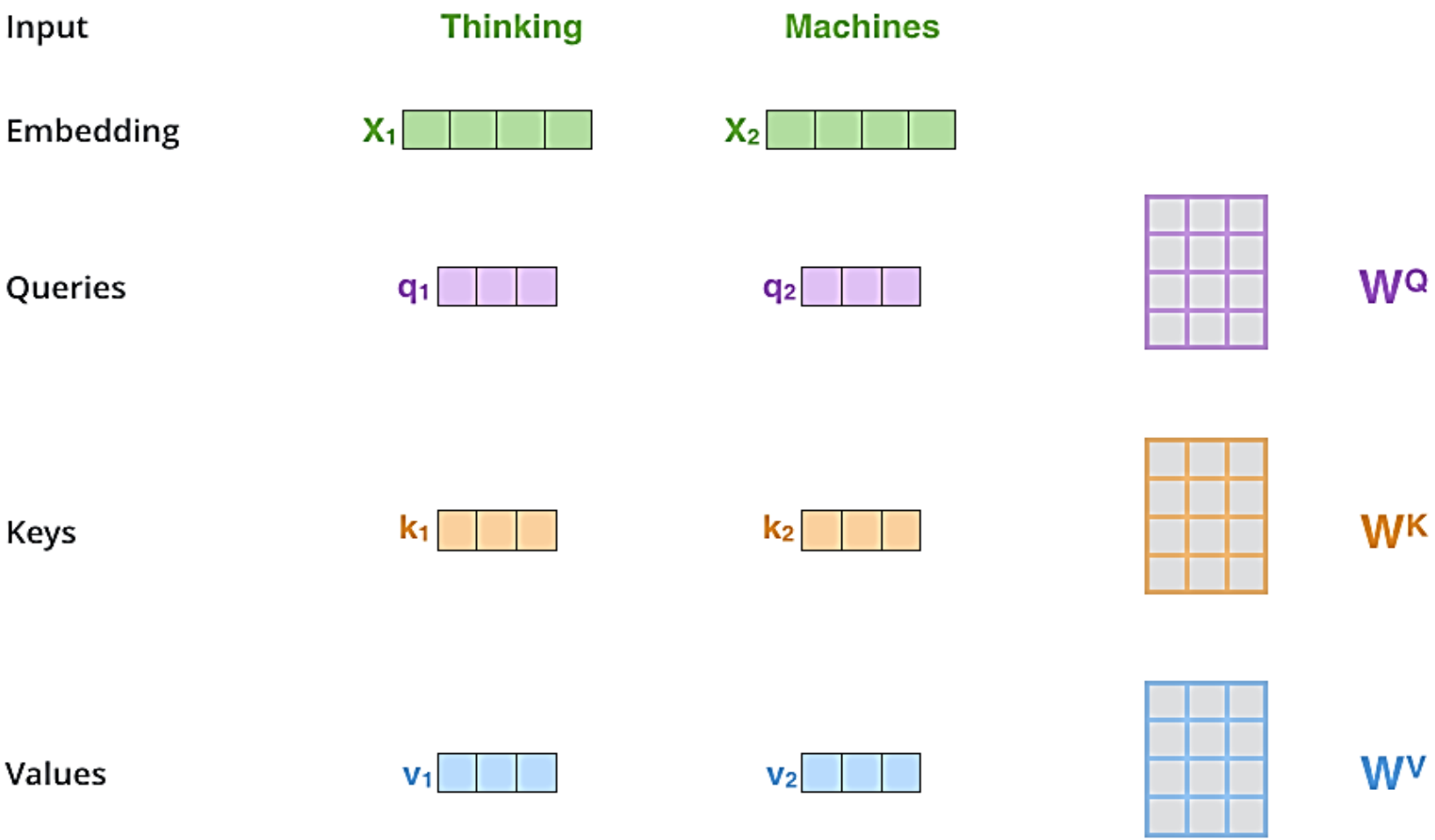
Attention weights: $A = softmax(E, \dim = 1) \ (Shape: N_" \times N_")$

Output vectors: $Y = AV \ (Shape: N_" \times D_*) Y_{\%} = \Sigma_' A_{\%\&} V_'$

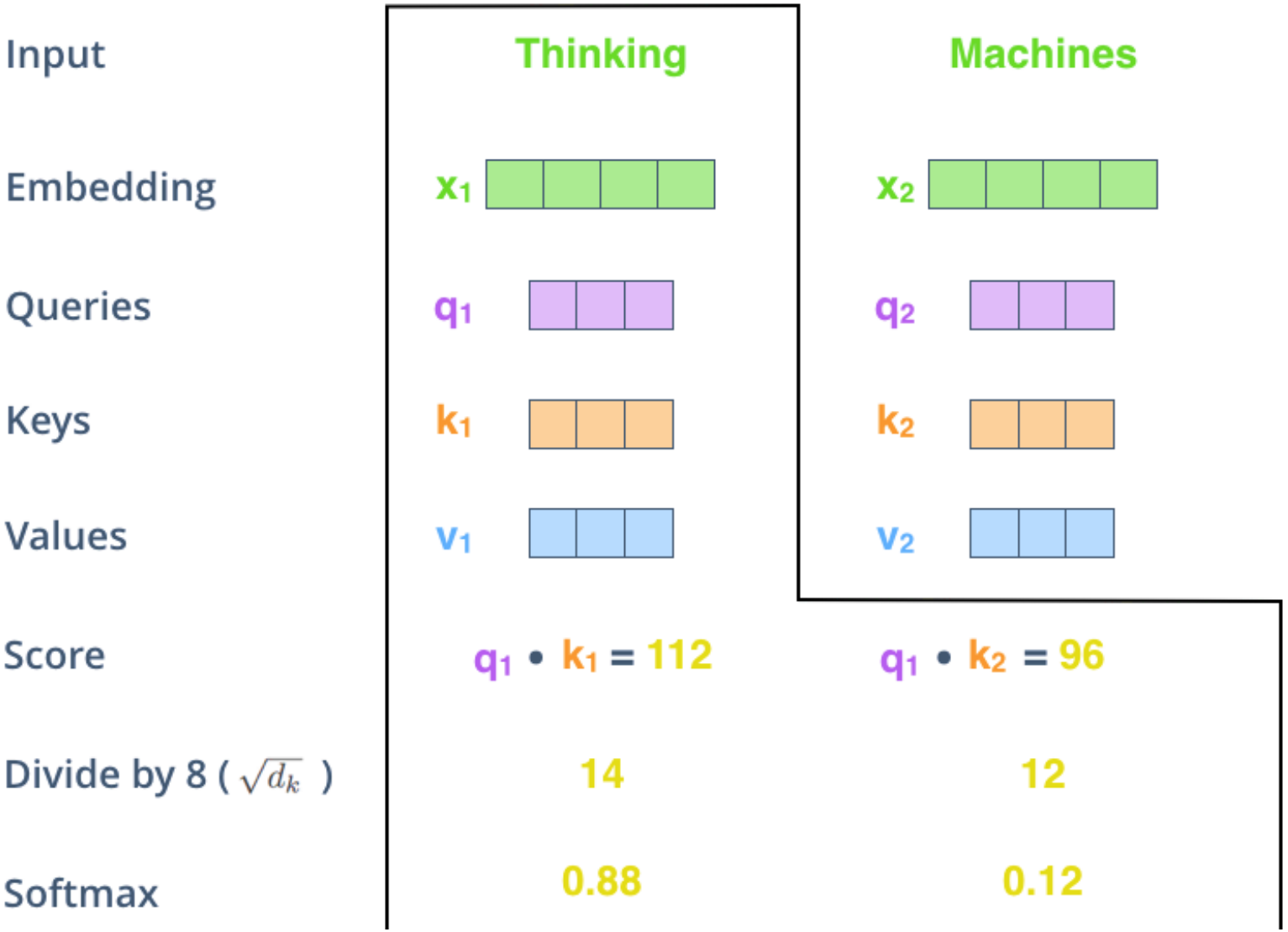


Blank Slide

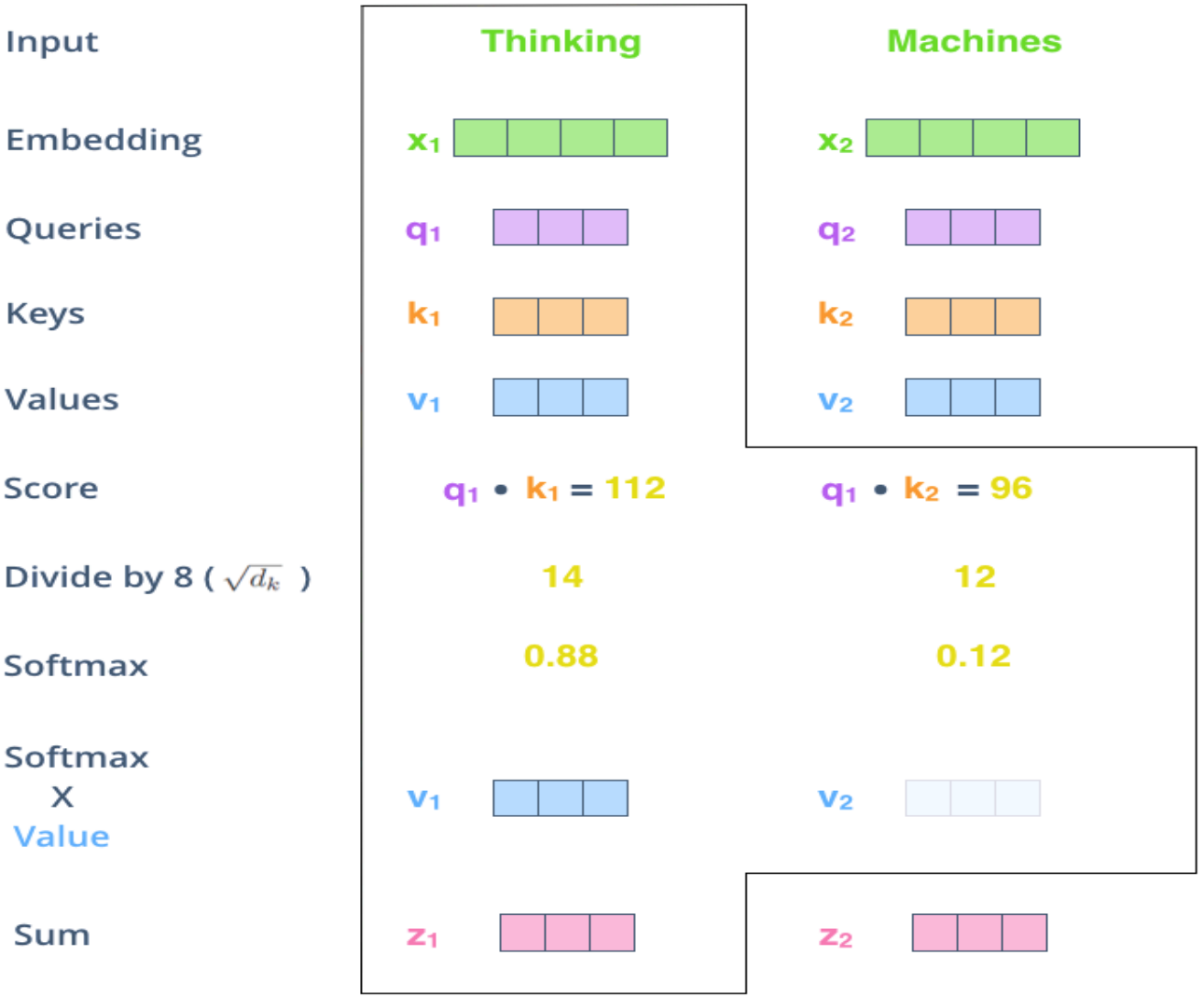
Self Attention: Step 1



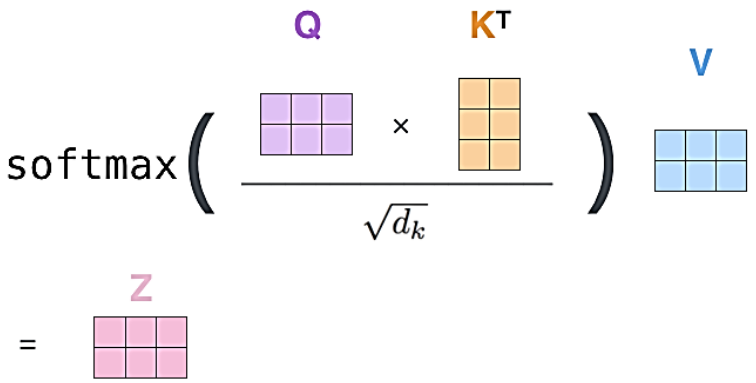
Self Attention: Step 2



Self Attention: Step 3

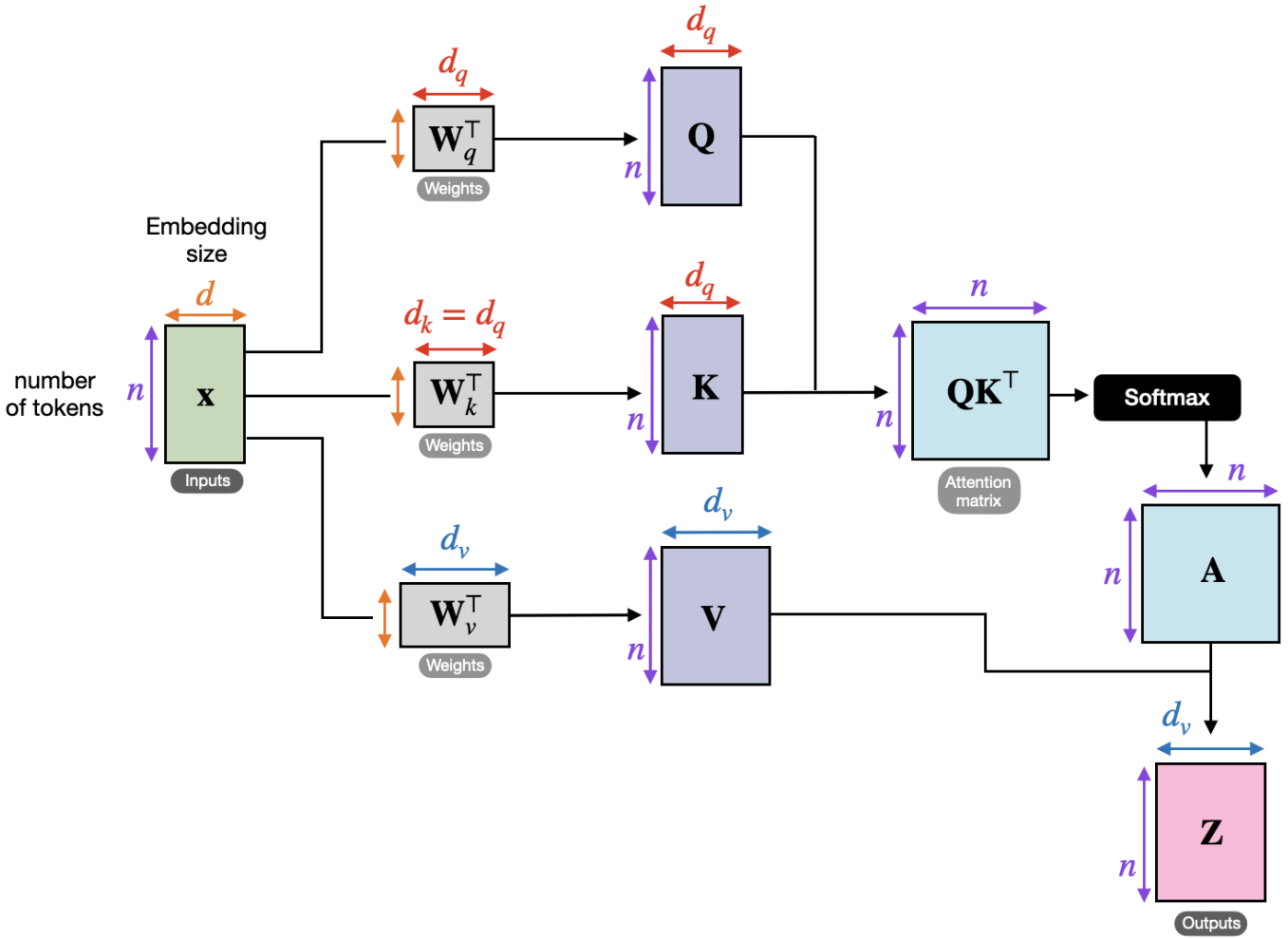


Matrix View



Blank Slide

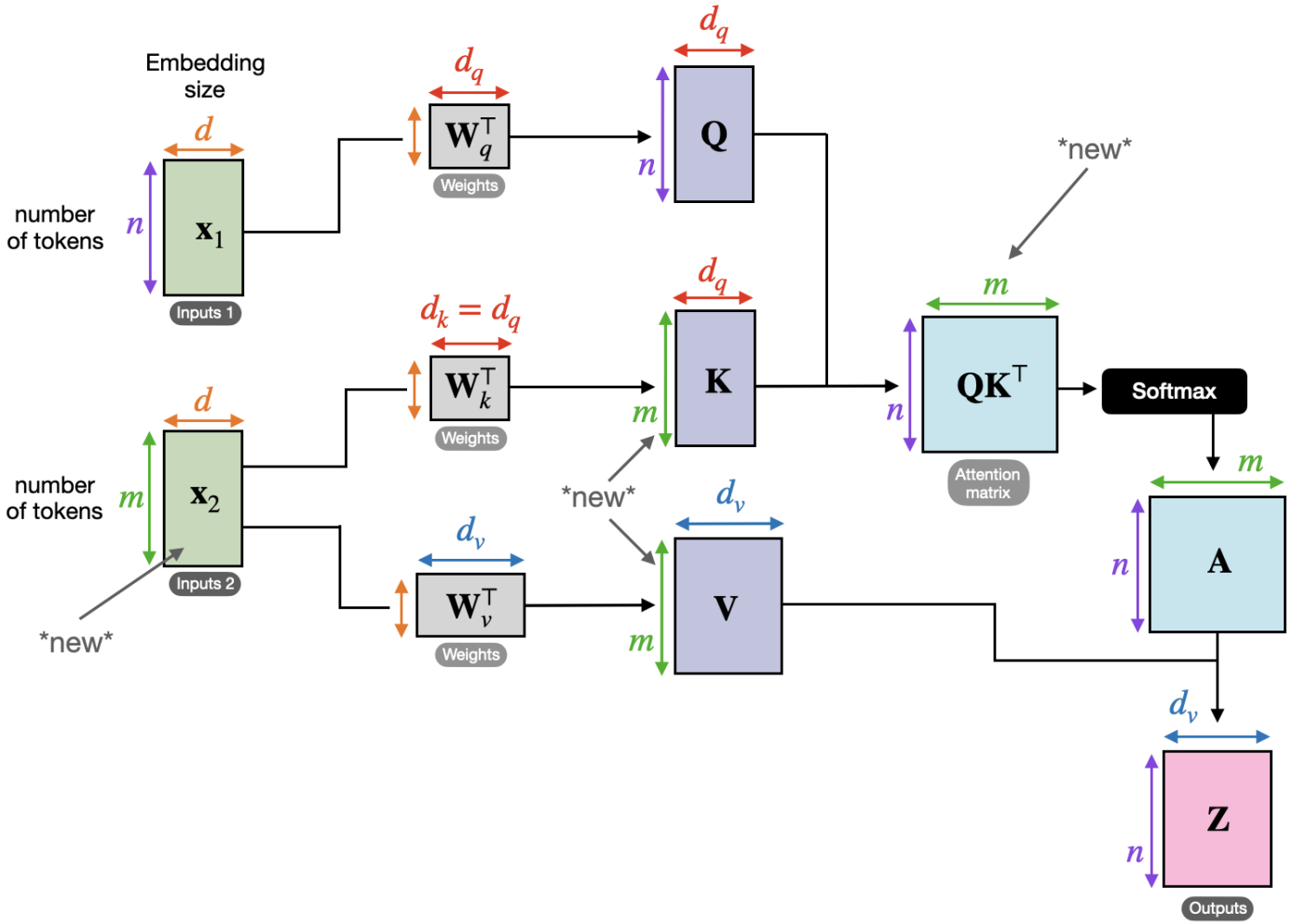
Self Attention: Computation

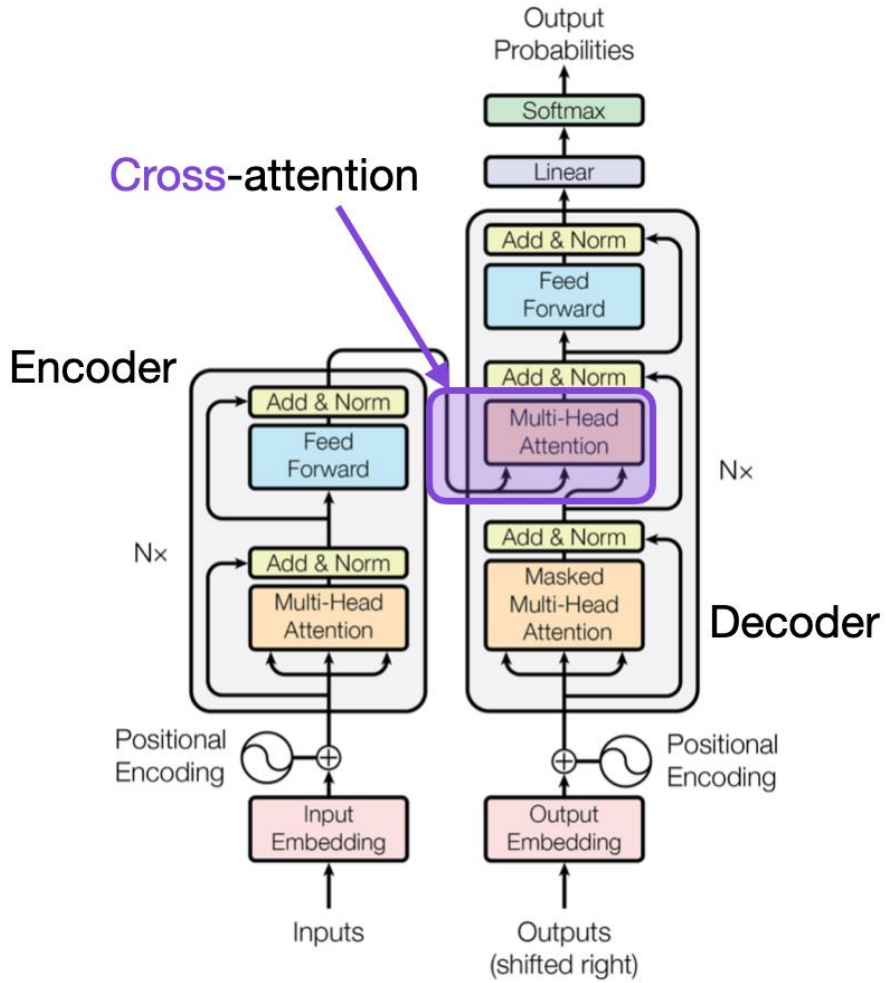


Example of Cross-Attention

- Task:
  - Given an Image (a set of patches), create a sentence
- Need of attention:
  - At any stage of creating a word, attend more to the region that is relevant

Cross Attention



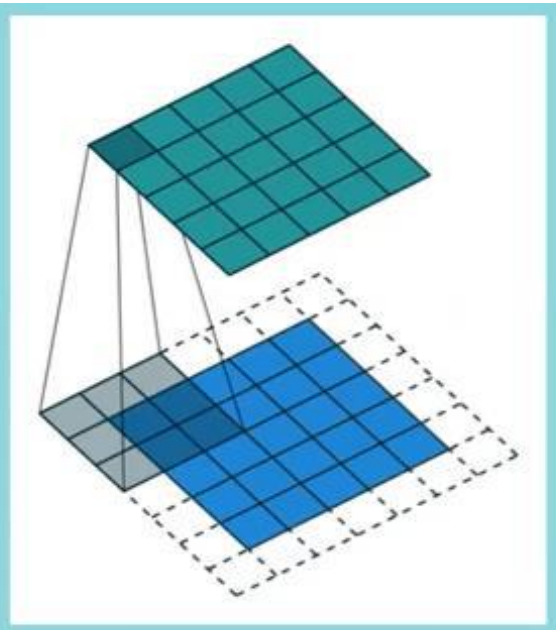


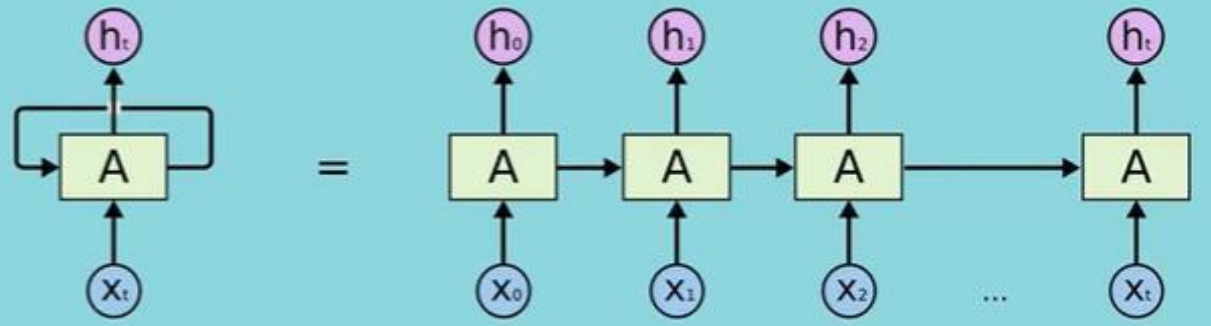
Source: "Attention Is All You Need" (https://arxiv.org/abs/1706.03762)

Example: Note that the queries usually come from the decoder, and the keys and values usually come from the encoder.

III. Discussions

CNN, RNN and Tranformers





Pro: Compute in Parallel
Con: Finite Horizon

Pro: Infinite Horizon
Con: Compute Serially only

Attention

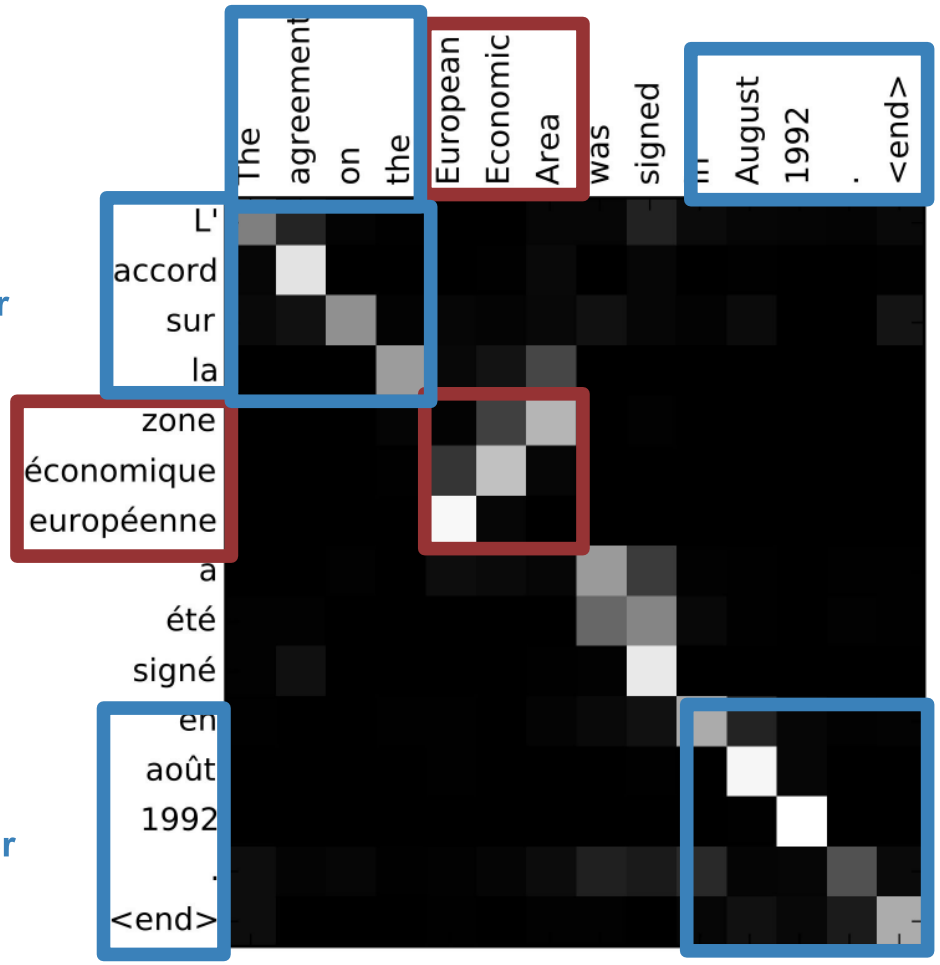
Visualization

Example: English to French translation

Input: "**The agreement on the European Economic Area** was signed **in August 1992**."

Output: "**L'accord sur la zone économique européenne** a été signé **en août 1992**."

Visualize attention weights $a_{t,i}$



Diagonal attention means words correspond in order

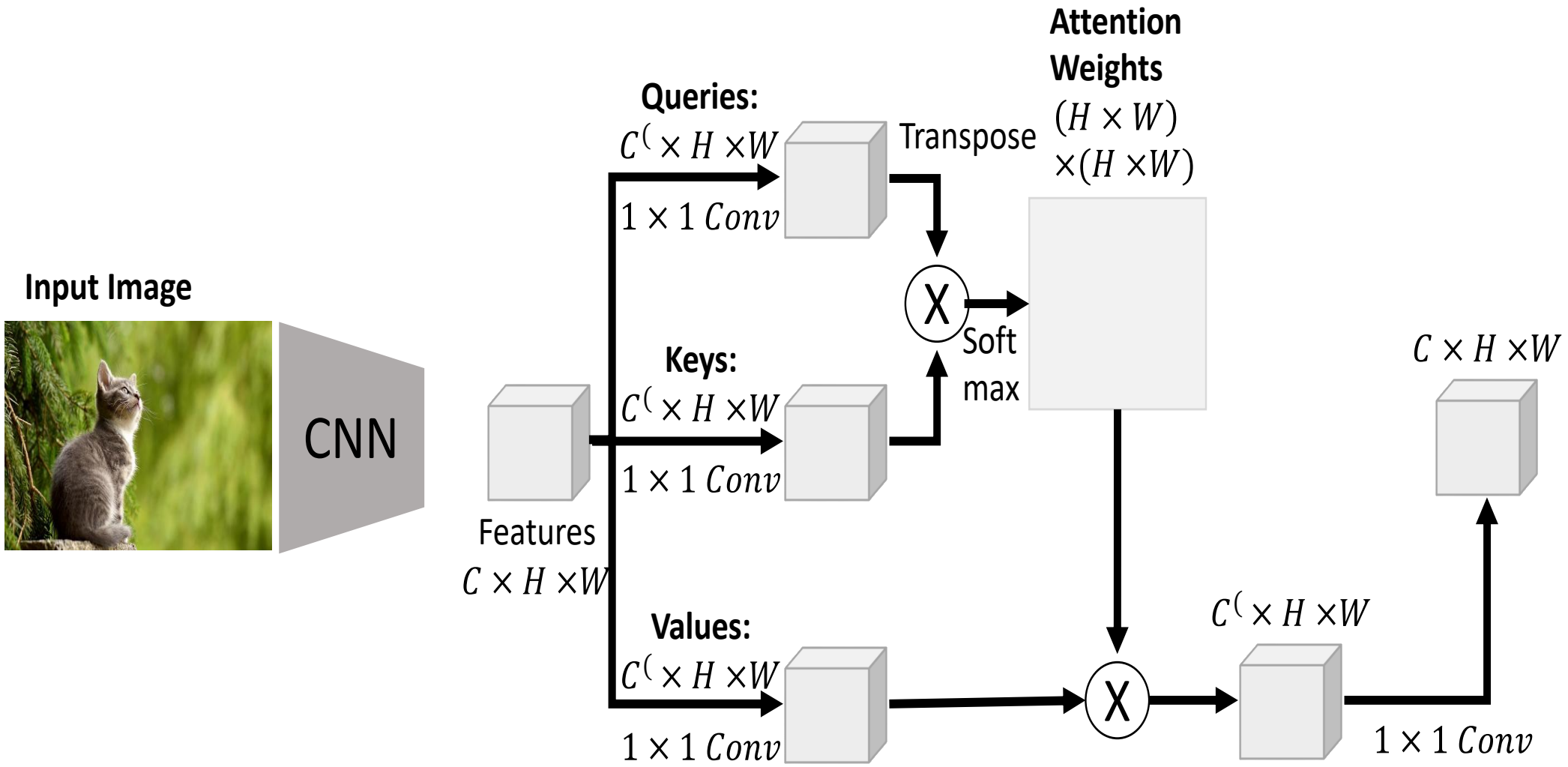
Attention figures out different word orders

Diagonal attention means words correspond in order

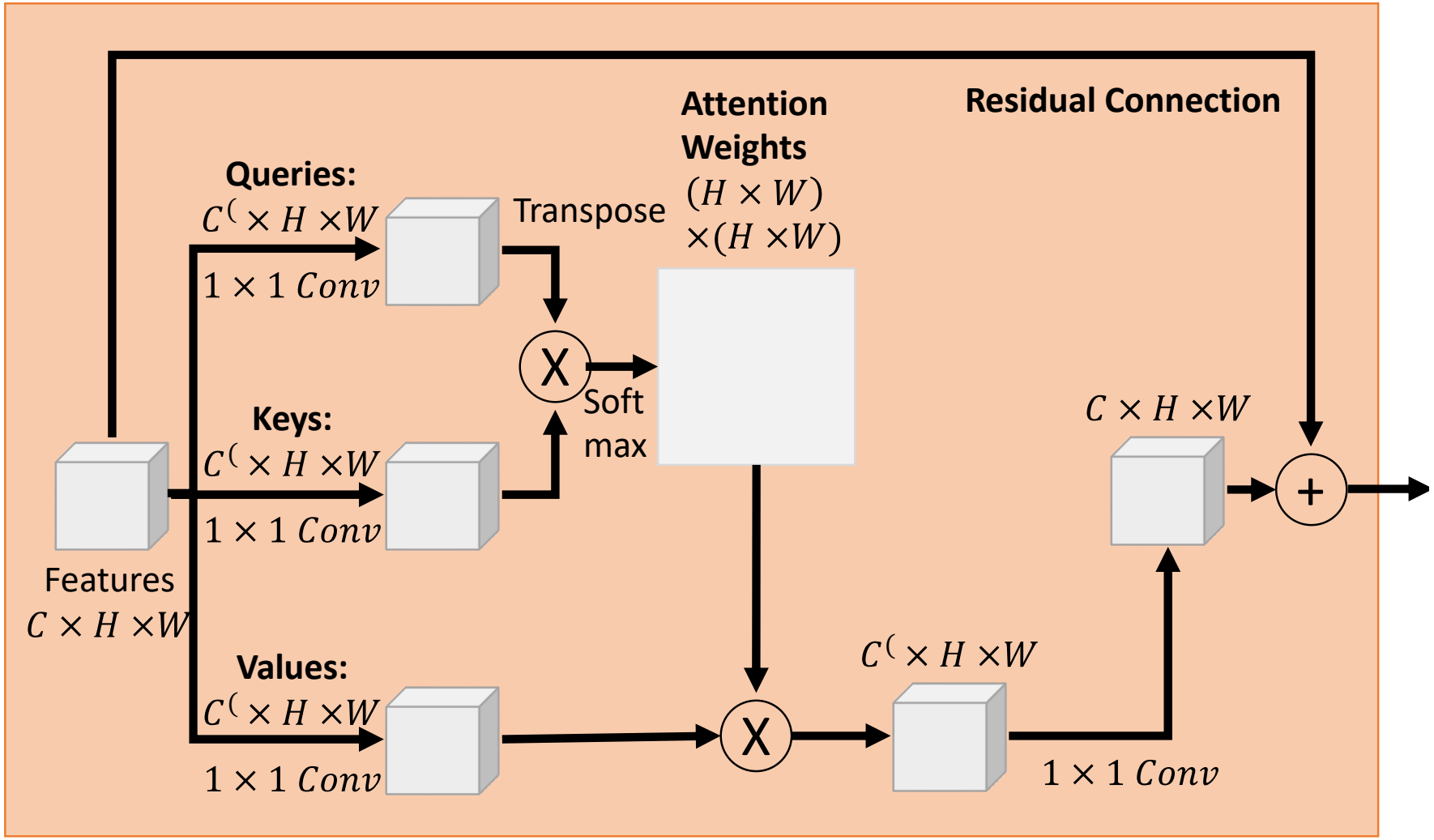
Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Blank Slide

CNN with Attention

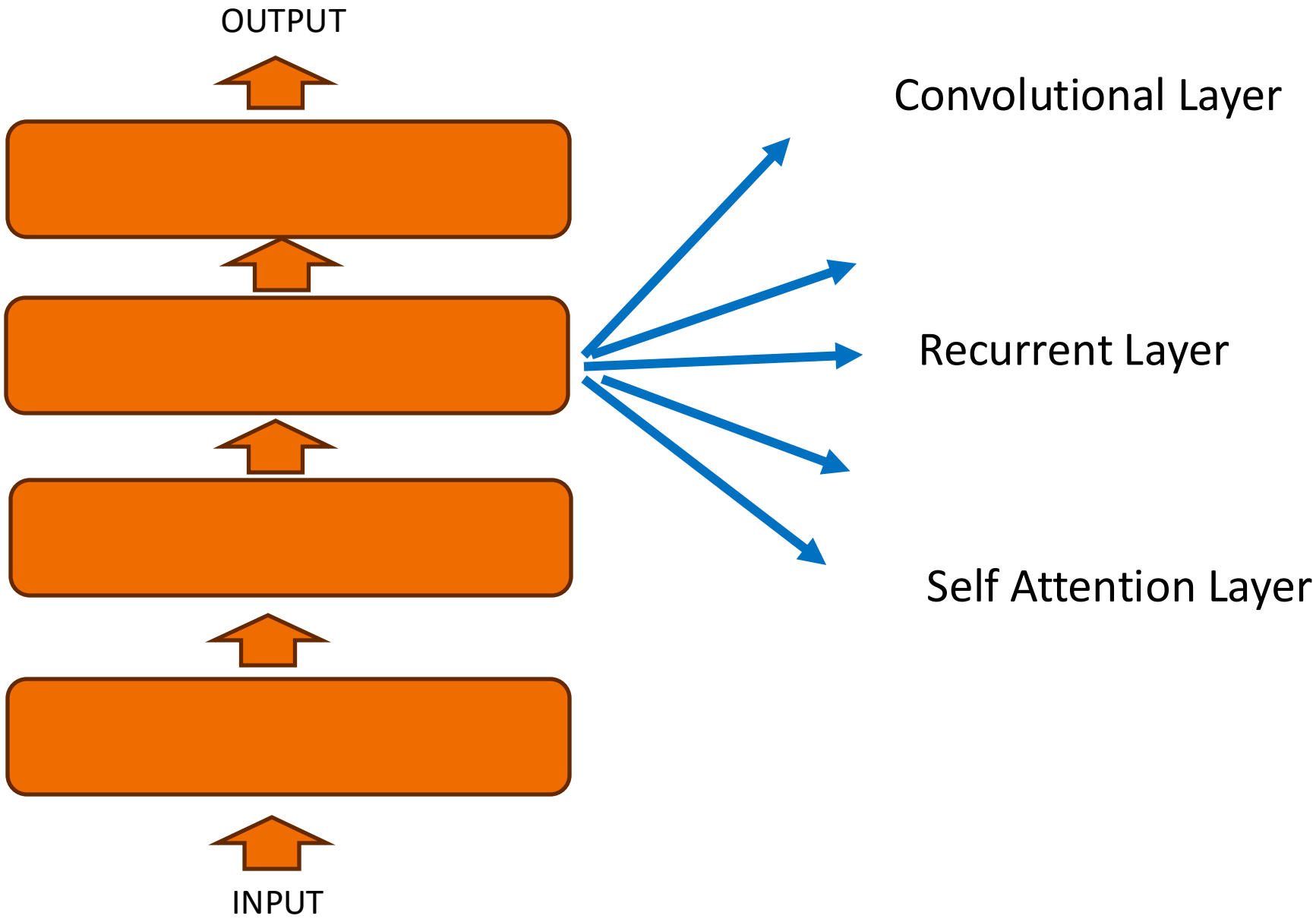


CNN with Attention



Self Attention Module

Self Attention Layer



Convolutional Layer

Recurrent Layer

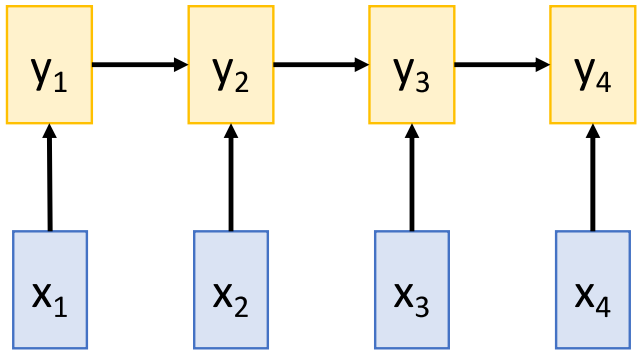
Self Attention Layer

Self Attention: Comments

- Self attention learns the relationship between elements in a sequence.
  - say between words in a sentence
- Self Attention Vs Convolution
  - Filters are dynamically calculated instead of static filters
  - SA is invariant to changes in the input points
  - SA can operate on irregular inputs
- SA allows to learn global and local features
  - Hierarchical feature learning by cascading

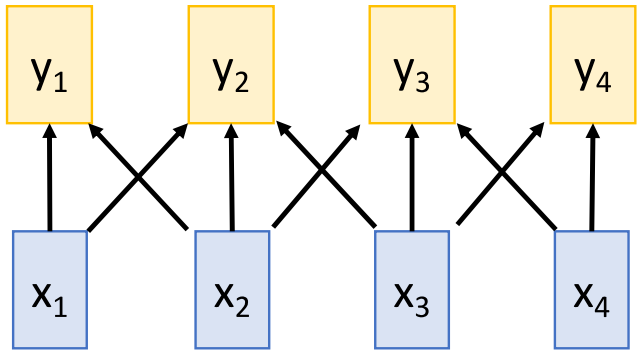
Conceptual Comparison

Recurrent Neural Network



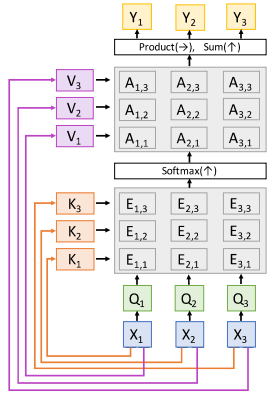
- Works on Ordered Sequences
- **Long range dependence (in the past)**
- **Not parallelizable**

1D Convolution



- Works on Multidimensional grids
- **Short visibility; Stacking helps**
- **Highly parallelizable**

Self-Attention



- **Works on Sets and Sequences**
- **Long range dependence**
- **Parallelizable**
- **Memory Intensive**

Thanks!!

Questions?